

UNIT-2

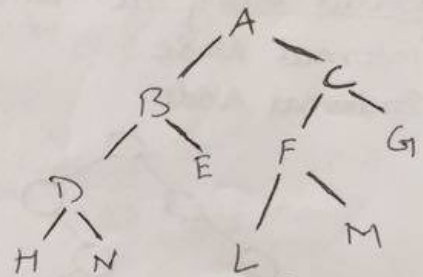
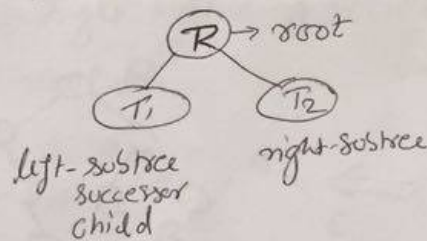
(7)

TREE: Tree is a non-linear data structure used mainly to represent hierarchical relationship between elements.

BINARY TREE:

A binary tree T is defined as a finite set of elements, called nodes, such that

- T is empty (called null tree or empty tree), or,
- T contains a distinguished node R , called the root of T , and the remaining nodes of T form an ordered pair of disjoint binary trees T_1 & T_2 .



Binary Tree

* Nodes with no successor are called terminal nodes.

Binary tree T & T' are said to be similar if they have the same structure or shape. The trees are said to be copies if they have same structure & same content at corresponding nodes.

Q $E = (a-b) / ((c*d) + e)$

Terminology:

- ① Left successor
- ② Right successor
- ③ Parent
- ④ Child
- ⑤ Siblings
- ⑥ Decendants
- ⑦ Ancestors

⑧ Edge/Path: The line drawn from a node N to a successor is called an edge. Sequence of consecutive edges is called a path.

⑨ Level: Each node in a binary tree is assigned a level number, such that root R of the tree T is assigned level number 0 & so on.

⑩ Depth(Height): The depth of a tree is the maximum no. of nodes in a branch of T.

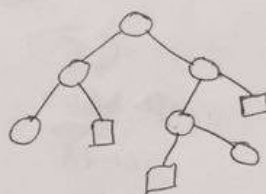
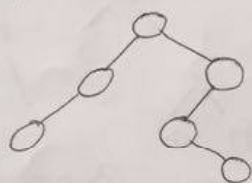
Complete Binary Tree: The left and right children of the node K are $(2 \times K)$ & $(2 \times K + 1)$ resp. & the parent node of K is $\lfloor K/2 \rfloor$, the depth of a complete tree is given by

$$D_n = \lfloor \log_2 n + 1 \rfloor$$

Extended Binary Tree: This tree has 0, 2 children only. (2-Tree).

→ Internal Node

→ External Node



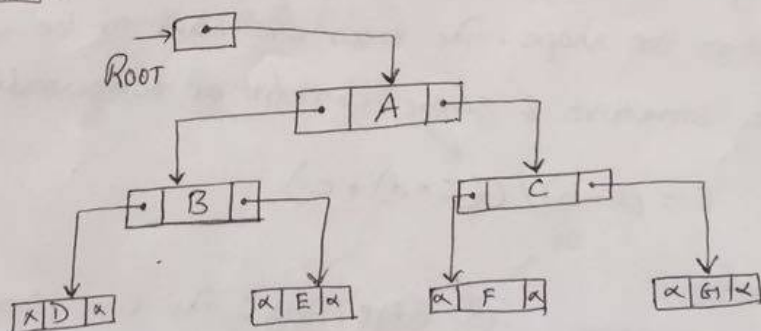
Representing In Memory:-

→ Linked Representation:-

① INFO[K]

② LEFT[K]

③ RIGHT[K]



⑧ TRAVERSING BINARY TREES:

① PREORDER

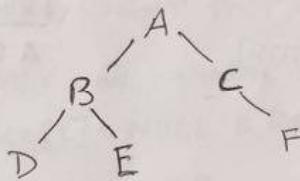
- ↳ Process the root R.
- ↳ Traverse the left subtree of R in preorder.
- ↳ Traverse the right " " R " " .

② INORDER

- ↳ Traverse the left subtree of R in Inorder.
- ↳ Process the root R.
- ↳ Traverse " right " " R " " .

③ POSTORDER

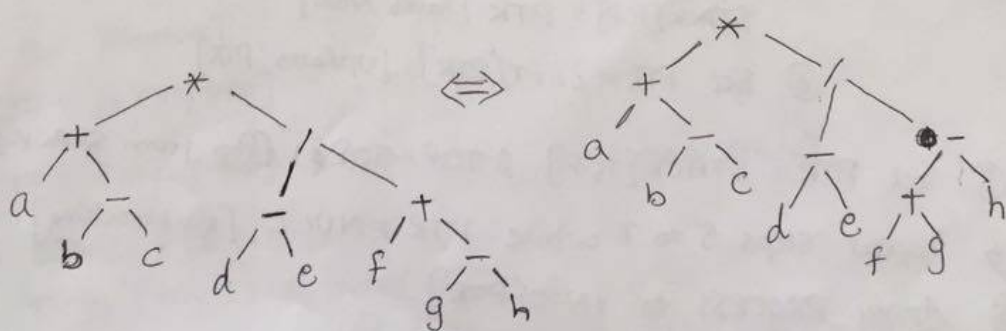
- ↳ Traverse the left subtree of R in Postorder.
- ↳ Traverse the right subtree of R in Postorder.
- ↳ Process the root R.



Q: $[a + (b - c)] * [(d - e) / (f + g - h)]$

Preorder: $* + a - b c / - d e \bar{\bullet} \bar{\bullet} f g h$

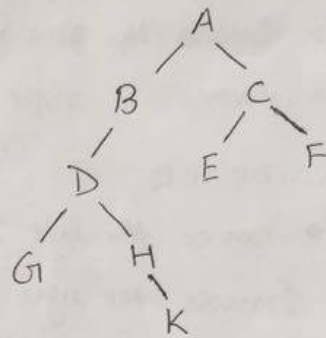
Postorder: $a b c - + d e - f g + h - / *$



Algo PREORD (INFO, LEFT, RIGHT, ROOT)

T $\rightarrow$ Binary tree, PROCESS, STACK $\rightarrow$ hold addresses of nodes, temp.

- ① Initially push NULL onto STACK & initialize PTR
Set TOP = ~~0~~ 1, STACK[1] = NULL, PTR = ROOT.
- ② Repeat steps 3 to 5 while PTR $\neq$ NULL
- ③ Apply PROCESS to INFO[PTR]
- ④ IF RIGHT[PTR] $\neq$ NULL, then:
Set TOP = TOP + 1
STACK[TOP] = RIGHT[PTR]
End of IF
- ⑤ IF LEFT[PTR] $\neq$ NULL, then:
Set PTR = LEFT[PTR]
ELSE (POP FROM STACK)
Set PTR = STACK[TOP]
TOP = TOP - 1
End of IF
- ⑥ Exit.



PREORDER:

A B D G H K C E F.

Algo INORD (INFO, LEFT, RIGHT, ROOT)

- ① Set TOP = 1, STACK[1] = NULL, PTR = ROOT
- ② Repeat while PTR $\neq$ NULL [Push left-most path onto STACK]
 - Ⓐ Set TOP = TOP + 1
STACK[TOP] = PTR [Saves Node]
 - Ⓑ Set PTR = LEFT[PTR] [Updates PTR]
- ③ Set PTR = STACK[TOP] & TOP = TOP - 1 [Pop from STACK]
- ④ Repeat steps 5 to 7 while PTR $\neq$ NULL [Backtracking]
- ⑤ Apply PROCESS to INFO[PTR]

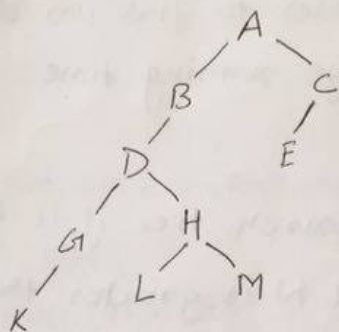
⑥ If $\text{RIGHT}[\text{PTR}] \neq \text{NULL}$

① Set $\text{PTR} = \text{RIGHT}[\text{PTR}]$

② Go To Step 2. [End of IF str]

⑦ Set $\text{PTR} = \text{STACK}[\text{TOP}]$ & $\text{TOP} = \text{TOP} - 1$
[End of Step 4 loop].

⑧ Exit



INORDER:
 $\Rightarrow K G D L H M B A E C$

POSTORDER TRAVERSAL:

① $\text{POSTORD}(\text{INFO}, \text{LEFT}, \text{RIGHT}, \text{ROOT})$

① Push NULL onto STACK and initialize PTR

Set $\text{TOP} = 1$, $\text{STACK}[1] = \text{NULL}$ & $\text{PTR} = \text{ROOT}$

② [Push left-most path onto STACK]

Repeat Steps 3 to 5 while $\text{PTR} \neq \text{NULL}$

③ Set $\text{TOP} = \text{TOP} + 1$ & $\text{STACK}[\text{TOP}] = \text{PTR}$

[Pushes PTR onto STACK]

④ If $\text{RIGHT}[\text{PTR}] \neq \text{NULL}$, then:

Set $\text{TOP} = \text{TOP} + 1$ & $\text{STACK}[\text{TOP}] = \text{RIGHT}[\text{PTR}]$

[End of IF structure]

⑤ Set $\text{PTR} = \text{LEFT}[\text{PTR}]$

⑥ Set $\text{PTR} = \text{STACK}[\text{TOP}]$ & $\text{TOP} = \text{TOP} - 1$ [POP from STACK]

⑦ Repeat while $\text{PTR} > 0$

(a) Apply PROCESS to $\text{INFO}[\text{PTR}]$

(b) Set $\text{PTR} = \text{STACK}[\text{TOP}]$ & $\text{TOP} = \text{TOP} - 1$ [POP node from STACK]

End of loop.

⑧ If $PTR < 0$ then:

(a) Set $PTR = -PTR$

(b) Go To Step ②

End of IF str.

⑨ Exit.

Binary Search Tree:- A BST enables to find an element with an average running time:

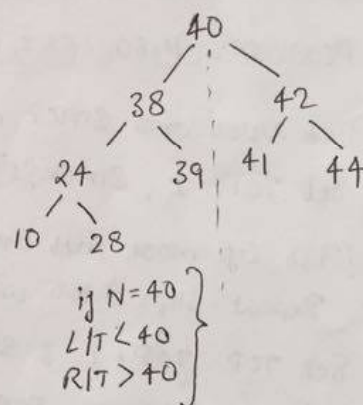
$$f_n = O(\log_2 n).$$

A binary tree becomes a binary search tree if it has the following property: The value at N is greater than every value in the left subtree of N & smaller than every value in the right subtree of N .

FIND(INFO, LEFT, RIGHT, ROOT, ITEM, LOC, PAR)

Three Cases:-

- 1) $LOC = NULL$ & $PAR = NULL$, tree empty.
- 2) $LOC \neq NULL$ & $PAR = NULL$, ITEM is the root.
- 3) $LOC = NULL$ & $PAR \neq NULL$, ITEM not in T, can be added to T.



Algo:-

- ① If $ROOT = NULL$, Set $LOC = NULL$ & $PAR = NULL$ & Return.
- ② If $ITEM = INFO[ROOT]$, Set $LOC = ROOT$ & $PAR = NULL$ & Return.
- ③ If $ITEM < INFO[ROOT]$, then:
Set $PTR = LEFT[ROOT]$, $SAVE = ROOT$
ELSE
Set $PTR = RIGHT[ROOT]$, $SAVE = ROOT$
- ④ Repeat steps 5 & 6 while $PTR \neq NULL$
- ⑤ If $ITEM = INFO[PTR]$, Set $LOC = PTR$ & $PAR = SAVE$ & Return.

⑥ IF $ITEM < INFO[PTR]$
 Set $SAVE = PTR$ $PTR = LEFT[PTR]$
 ELSE
 Set $SAVE = PTR$ $PTR = RIGHT[PTR]$
 END of IF
 END of Step 4

⑦ [Search unsuccessful] Set $LOC = NULL$ & $PAR = SAVE$.

⑧ Exit.

INSERTION:-

INSBST (INFO, LEFT, RIGHT, ROOT, AVAIL, ITEM, LOC)

① Call FIND algo.

② IF $LOC \neq NULL$, then EXIT

③ ELSE: (a) IF $AVAIL = NULL$, then O/F & EXIT

(b) Set $NEW = AVAIL$, $AVAIL = LEFT[AVAIL]$ & $INFO[NEW] = ITEM$

(c) Set $LOC = NEW$, $LEFT[NEW] = NULL$ & $RIGHT[NEW] = NULL$.

④ IF $PAR = NULL$, then

Set $ROOT = NEW$

ELSE-IF $(ITEM < INFO[PAR])$, then:

Set $LEFT[PAR] = NEW$

ELSE

Set $RIGHT[PAR] = NEW$

END of IF.

⑤ EXIT.

DELETION: For deletion there are three cases

a) The node N does not have any child.

b) The node N has only one child (left/right).

c) The node N has both left & right child.

Algo:

CASE A (INFO, LEFT, RIGHT, ROOT, LOC, PAR)

① If $LEFT[LOC] = NULL$ & $RIGHT[LOC] = NULL$ then:

Set $CHILD = NULL$

Else-If $LEFT[LOC] \neq NULL$ then:

Set $CHILD = LEFT[LOC]$

Else

Set $CHILD = RIGHT[LOC]$

END of IF

② If $PAR \neq NULL$, then:

If $LOC = LEFT[PAR]$ then:

Set $LEFT[PAR] = CHILD$

ELSE

Set $RIGHT[PAR] = CHILD$

ELSE

Set $ROOT = CHILD$

③ RETURN.

CASE B (INFO, LEFT, RIGHT, ROOT, LOC, PAR)

This procedure will delete the Node N at location LOC , where N has two children. The pointer PAR gives the location of the parent N or $PAR = NULL$. SUC will be $INORD(N)$ & $PARSUC$ will be PAR of $SUC(N)$.

① Set $PTR = RIGHT[LOC]$ & $SAVE = LOC$

② Repeat while $LEFT[PTR] \neq NULL$

Set $SAVE = PTR$ & $PTR = LEFT[PTR]$.

③ Set $SUC = PTR$ & $PARSUC = SAVE$

④ Call CASE A (INFO, LEFT, RIGHT, ROOT, SUC, PARSUC) [Delete INORD]

⑤ Replace Node N by its INORD SUC .

⑥ If $PAR \neq NULL$ then: If $LOC = LEFT[PAR]$

~~IF~~ $LEFT[PAR] = SUC$

ELSE:

Set $RIGHT[PAR] = SUC$

ELSE:

Set $ROOT = SUC$.

⑦ Set $LEFT[SUC] = LEFT[LOC]$

$RIGHT[SUC] = RIGHT[LOC]$

⑧ RETURN.

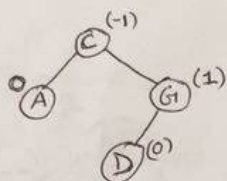
AVL Tree:

(11)

A binary tree, depending upon the elements encountered may result in a right-skew or left-skew & thereby increasing its complexity to be $O(n)$. AVL (Adelson-Velskii-Landis) tree is balanced tree which maintains the complexity $O(\log n)$ in the worst case.

Definition: Let us consider a tree T , let T^L & T^R are left & right sub-trees and similarly $h(T^L)$ & $h(T^R)$ are the heights. The value $|h(T^L) - h(T^R)|$ is known as Balance Factor & for a tree to be AVL tree it can be 0, 1, -1.

Eg:



An AVL search tree is a binary tree which is an AVL tree.

INSERTION:-

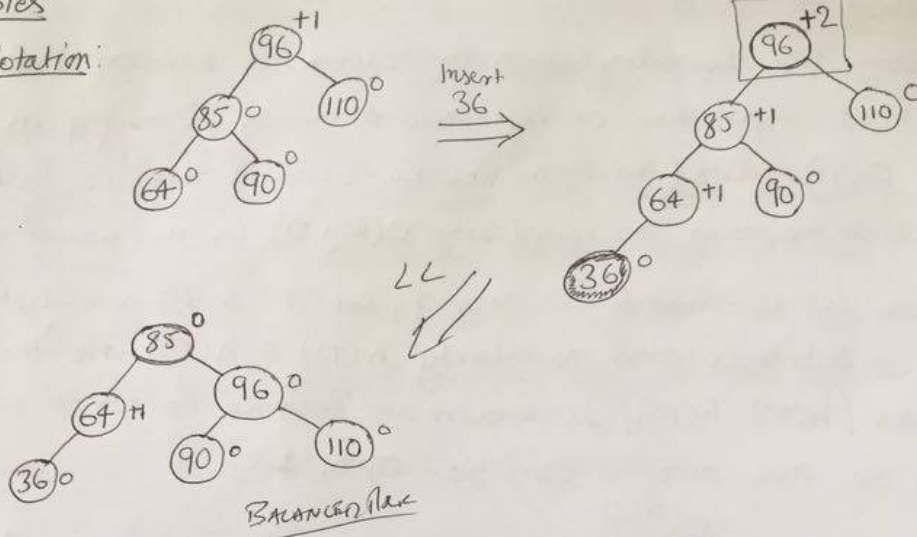
Insertion in an AVL tree is similar to the binary tree, however if the insertion affects the Balance Factor of a node, the balance is restored by performing rotations.

To perform rotation a node A is specified whose balance factor is neither 0, 1 or -1 and is the nearest ancestor to the inserted node, on the path from inserted node to the root. The rebalancing rotations are classified as:

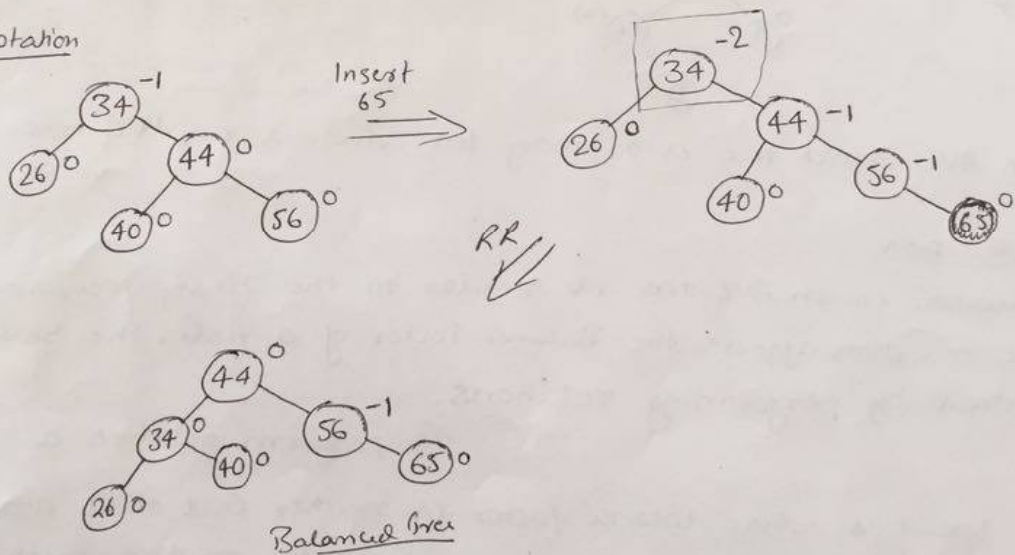
- 1) LL: Inserted node is in left-subtree of left-subtree of Node A.
- 2) RR: " " right-subtree " right-subtree Node A.
- 3) LR: " " left-subtree of right-subtree of Node A.
- 4) RL: " " right-subtree " left-subtree of Node A.

Examples

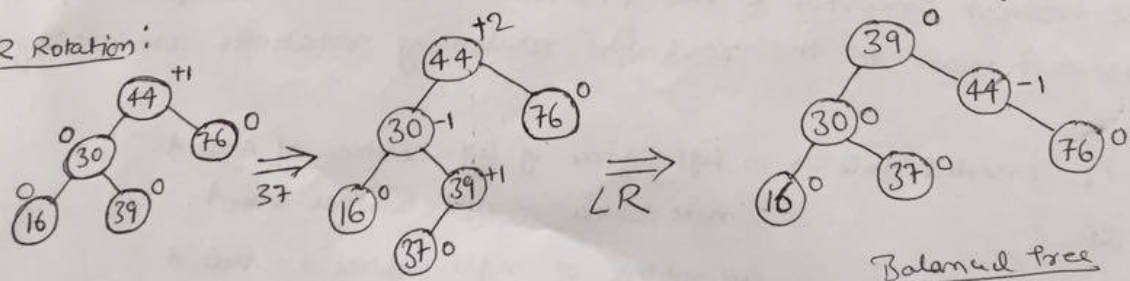
LL Rotation:



RR Rotation



LR Rotation:

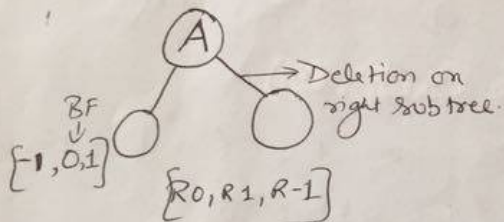


Q. Construct an AVL tree by inserting following elements ⁽¹²⁾
in order: 64, 1, 14, 26, 13, 110, 98, 85.

Deletion in AVL Tree:

Deletion of an element in AVL tree may result in an unbalanced Balance Factors, which needs to be ~~maintained~~ restored. This balancing is done by performing rotations to the left or right depending on whether the deletion occurred on the left or right subtree.

*
Explain: R0, R1, R-1 & L0, L1, L-1 rotations.



X — X